



Conceptos generales de arquitectura de software

Johnathan Calle, Ph.D.



Qué es arquitectura de software?

- Diseño y decisiones relacionadas a la estructura y comportamiento general del sistema
 - Implementación → estructuras de proyecto
 - Implementación → tipo de base de datos?
 - Tecnologías → lenguaje?
 - Diseño de sistema → monolítico?
 - Infraestructura → cloud?



Qué es arquitectura de software?

IEEE 1471-2000:

La Arquitectura de Software es la organización fundamental de un sistema en sus componentes, relaciones entre ellos, el ambiente y los principios que orientan su diseño y evolución.

Clements:

La Arquitectura de Software es a grandes rasgos, una vista del sistema que incluye los componentes principales del mismo, la conducta de esos componentes según se la percibe del resto del sistema y las formas en que los componentes interactúan y se coordinan para alcanzar la misión del sistema.

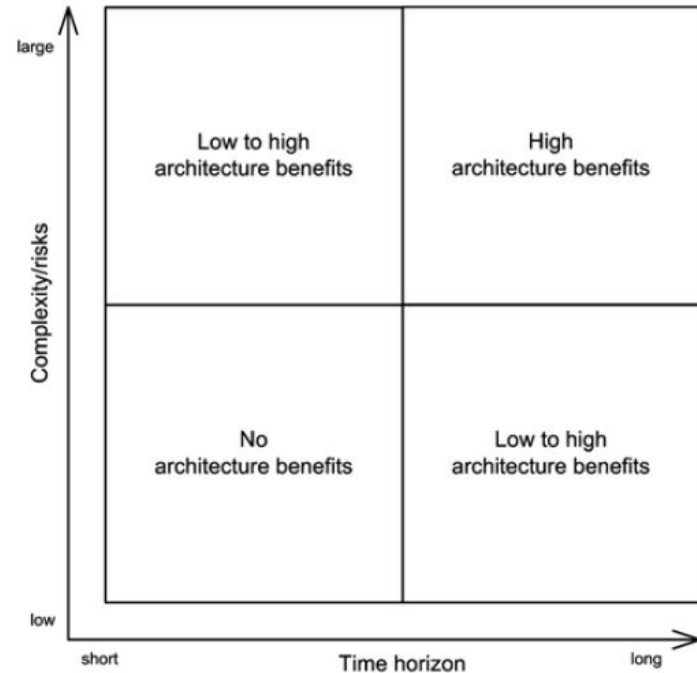


Qué es arquitectura de software?

- Diseño y decisiones relacionadas a la estructura y comportamiento general del sistema
 - Implementación → estructuras de proyecto
 - Implementación → tipo de base de datos?
 - Tecnologías → lenguaje?
 - Diseño de sistema → monolítico?
 - Infraestructura → cloud?
- **Diseño a nivel de implementación**

Qué es arquitectura de software?

Beneficios →





Algunas frases interesantes

- Architecture represents the significant design decisions that shape a system, where significant is measured by cost of change. —Grady Booch
- If you think good architecture is expensive, try bad architecture. —Brian Foote and Joseph Yoder
- Architecture is a hypothesis, that needs to be proven by implementation and measurement. —Tom Gilb
- The only way to go fast, is to go well. —Robert C. Martin



¿Por qué es importante la arquitectura de software?



Qué preguntas responde la arquitectura de software?

- En qué requisitos se basan la estructura y las decisiones?
- Cuáles son los bloques lógicos y físicos del sistema?
- Qué responsabilidades tienen estos bloques?
- Qué interfaces tienen estos bloques?
- Cómo se agrupan o distribuyen los bloques?
- Cuáles son las especificaciones y criterios para dividir el sistema en estos grandes bloques?



Qué preguntas responde la arquitectura de software?

- En qué requisitos se basan la estructura y las decisiones?
- Cuáles son los bloques lógicos y físicos del sistema?
- Qué responsabilidades tienen estos bloques?
- Qué interfaces tienen estos bloques?
- Cómo se agrupan o distribuyen los bloques?
- Cuáles son las especificaciones y criterios para dividir el sistema en estos grandes bloques?

→ Punto de partida: **Requisitos**



Para qué sirve la arquitectura de software?

- Plan de diseño para negociación de requisitos de sistema.
- Medio para establecer discusiones con clientes, desarrolladores y administradores.
- Dos formas de usar un modelo arquitectónico:
 - Como una forma de facilitar la discusión acerca del diseño del sistema.
 - Como una forma de documentar una arquitectura que se haya diseñado



Hasta dónde llega la arquitectura?

- La arquitectura va desde el análisis del dominio del problema de un sistema específico hasta la definición del sistema
 - Vista informal / formal
- No está presente en detalles a niveles de abstracción muy pequeños
 - Algoritmos
 - Clases individuales
 - **(Sin embargo, puede permear este nivel de abstracción)**
- Está presente a un nivel de abstracción superior
 - Subsistemas y componentes



Hasta dónde llega la arquitectura?

- La arquitectura va desde el análisis del dominio del problema de un sistema específico hasta la definición del sistema
 - Vista informal / formal
- No está presente en detalles a niveles de abstracción muy pequeños
 - Algoritmos
 - Clases individuales
 - **(Sin embargo, puede permear este nivel de abstracción)**
- Está presente a un nivel de abstracción superior
 - Subsistemas y componentes
- **Permite visualizar el sistema desde una perspectiva general → Abstracción**
- **Transversal al proceso → creación, entrega y operabilidad**



Motivación de la arquitectura de software

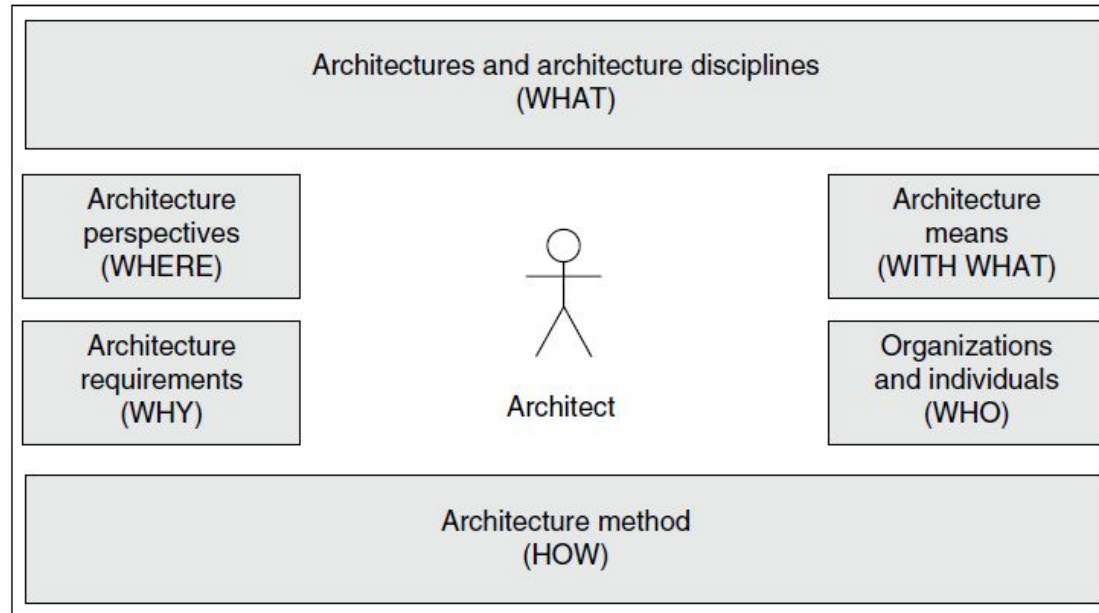
- Ambiente **variado** y **dinámico**
 - Nuevas herramientas, nuevos lenguajes, nuevos conceptos
- El rol de la **arquitecta**:
 - Entender el **contexto** y cómo puede variar para llegar a la solución adecuada
 - Entender los requisitos y proveer una solución **viable**
 - **Desarrollar el *architectural awareness***
 - Desarrollar habilidades alrededor de los diferentes contextos y estilos de arquitectura
 - El tiempo y la experiencia es clave aquí → es un proceso arduo



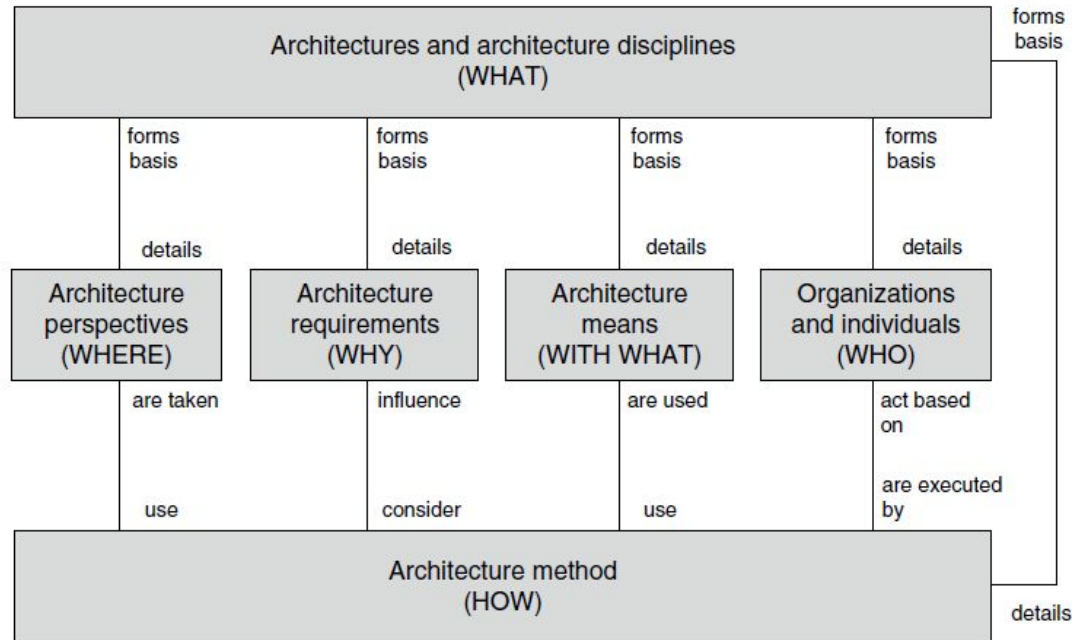
Motivación de la arquitectura de software

The goal of software architecture is to minimize the human resources required to build and maintain the required system.

Dimensiones de la arquitectura de software



Relaciones entre las dimensiones





Disciplinas de arquitectura (WHAT)

- Arquitectura de software
- Arquitectura de datos
- Arquitectura de integración
- Arquitectura de red
- Arquitectura de seguridad
- Arquitectura de gestión de sistema
- Arquitectura empresarial



Perspectivas de arquitectura (WHERE)

- Foco en perspectivas **específicas**
 - Una al tiempo → el desarrollo de software ya es lo suficientemente complejo
- Niveles de arquitectura
 - Disciplinas
- Modelos y vistas de arquitecturas
 - Abstracciones más generales y distintos puntos de vista de la arquitectura en cuestión



Requisitos de arquitectura (WHY)

- La arquitectura no es un fin por sí misma
 - Primero se diseña el sistema y como consecuencia surge la arquitectura
- La motivación principal de la arquitectura no es la elegancia
 - No es estética!
- **Es ética**
 - Valor agregado
 - De nada sirve cumplir con los requisitos funcionales pero no con los no funcionales
- **La arquitectura debe apoyar a todos los requisitos relacionados al sistema en desarrollo**



Requisitos de arquitectura (WHY)

- Sí, existen los requisitos funcionales y no funcionales, pero estos pueden tener características bastante específicas
 - Requisitos organizacionales
 - Requisitos de sistema
 - Requisitos de los bloques de construcción del sistema
 - Requisitos de tiempo de desarrollo del sistema
 - Requisitos de ejecución del sistema
 - Restricciones organizacional



Medios de arquitectura (WITH WHAT)

- Los medios cambiarán con base en los cambios del contexto
 - Industria
 - Alcance de problemas
 - Relevancia
- Principios de arquitectura
 - Se mantienen relevantes porque son fundamentales → separación de responsabilidades
- Conceptos básicos
 - Principios y paradigmas de diseño y programación
 - Orientación a objetos, a componentes
 - Desarrollo basado en modelos



Medios de arquitectura (WITH WHAT)

- Tácticas, estilos y patrones
 - Caja de herramientas
 - Reutilización
 - Se basan en principios de arquitectura
 - Implementaciones específicas
 - Las tácticas ayudan a tener una primera vista del problema
 - Un estilo documenta y comunica una manera probada y exitosa de estructurar un arquitectura
 - Un patrón de arquitectura describe un estilo de arquitectura usando una estructura general



Medios de arquitectura (WITH WHAT)

- Arquitecturas básicas
 - Computación en la nube
 - Flujo de datos
 - Arquitectura por capas
 - Arquitectura middleware
 - Orientada a servicios
 - etc.



Organizaciones e individuos (WHO)

- Stakeholders
- Comunicación y validación
- Refinamiento de requisitos y búsqueda de valor



El método en arquitectura de software (HOW)

- La arquitectura se debe usar como base fundamental para el sostenimiento del sistema
 - Para lograrlo, el arquitecto se apoya en medios de arquitectura
 - A diversos niveles de abstracción
 - A diversos niveles de comunicación → compañeros de equipo, líderes de proyecto, analistas, desarrolladores
 - Este proceso será exitoso si se mantiene en el tiempo y se puede replicar
- El método:
 - Crear la visión inicial del sistema
 - Entender los requisitos
 - Diseñar la arquitectura
 - Implementar la arquitectura
 - Comunicar la arquitectura



Quién es la Arquitecta de Software?

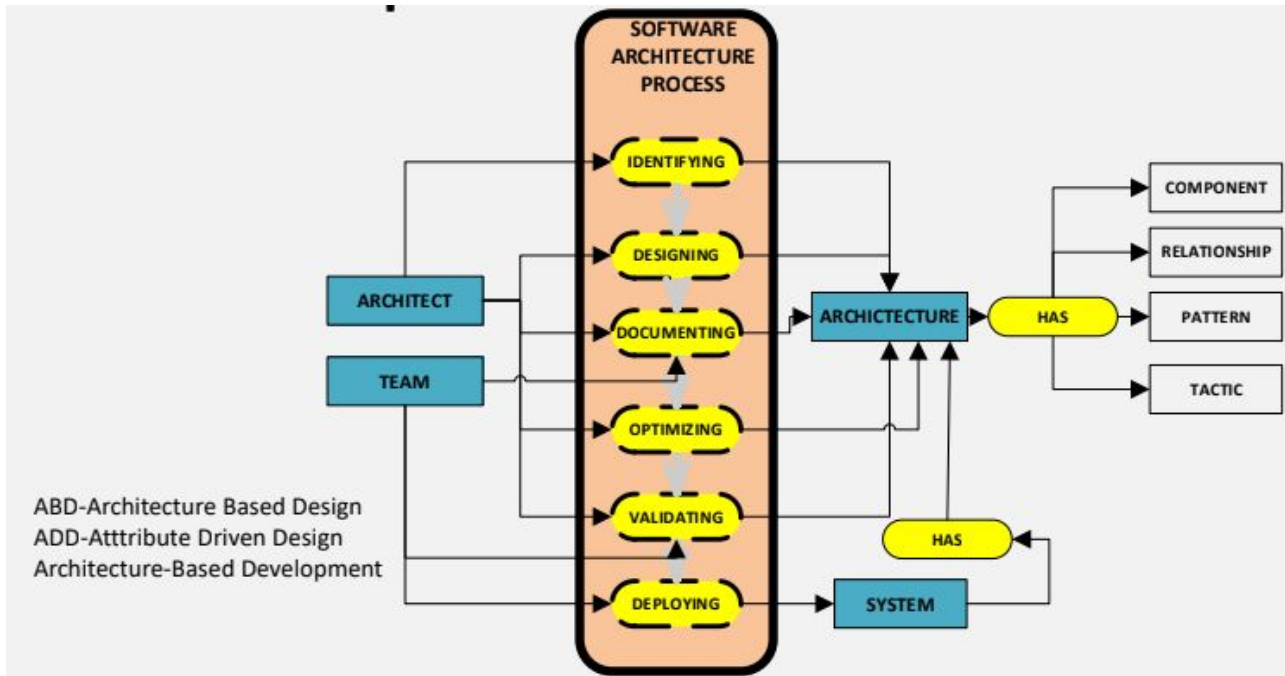
“A software architect is responsible for creating or selecting the most appropriate architecture for a system (or Systems), such that it suits the business needs, satisfies user requirements, and achieves the desired result under given constraints”



Quién es la Arquitecta de Software?

- Líder técnica.
- Combina su experiencia, conocimientos y habilidades, técnicas y no técnicas.
- Tiene conocimientos de diseño de arquitecturas de software, patrones de arquitectura y mejores prácticas.
- Es capaz de mitigar los riesgos y evaluar soluciones de manera que pueda seleccionar la adecuada para resolver un problema particular.
- Es competente en lenguajes, herramientas, marcos de trabajo y bases de datos que utiliza en sus sistemas de software.
- Tiene la amplitud de conocimientos para estar al tanto de las múltiples soluciones a un problema.
- Investiga sobre soluciones y sistemas.

Proceso de la arquitectura de software





Qué restricciones tengo?

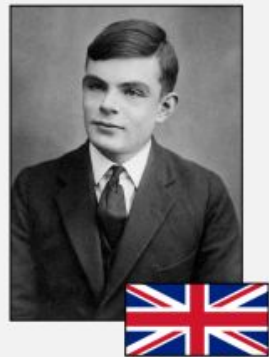
- Lenguaje de programación?
- Paradigma de programación?
- Framework?

Qué restricciones tengo?

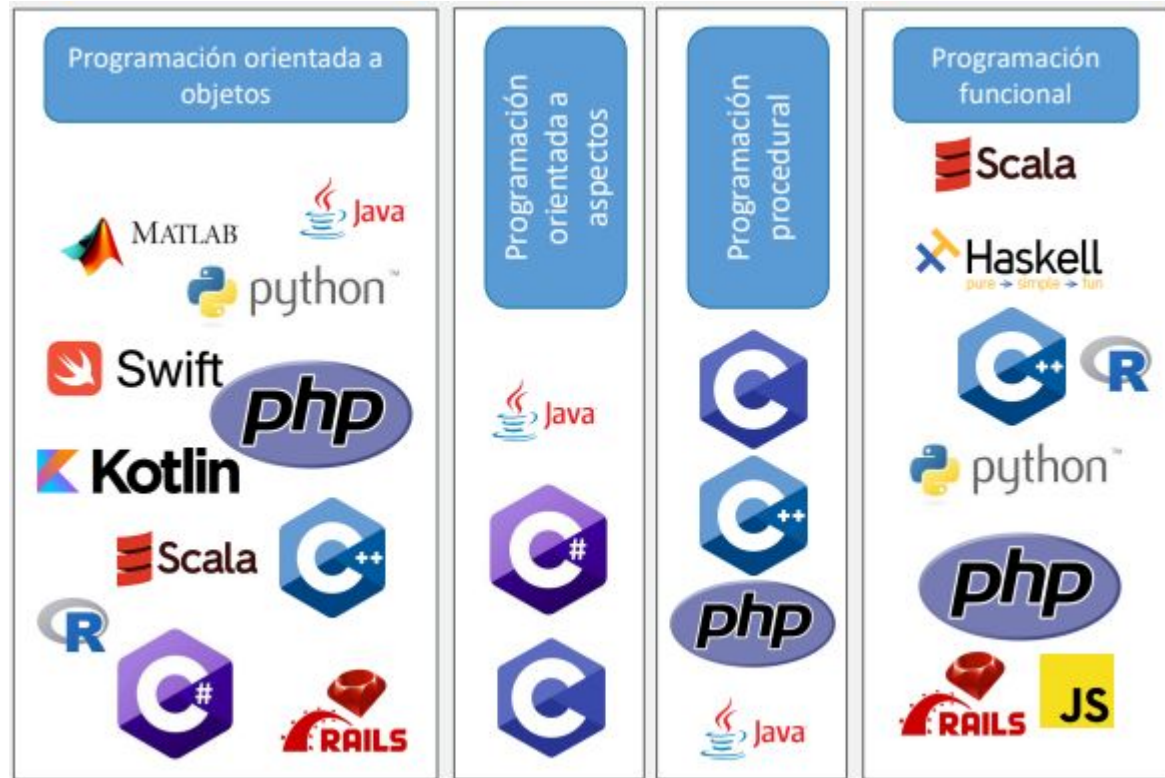
- Lenguaje de programación?
- Paradigma de programación?
- Framework?

La arquitectura de software empieza con código.

- En 1938, Alan Turing sentó las bases de lo que se convertiría en programación de computadoras. Para 1945, Turing estaba escribiendo programas reales en computadoras reales en código que reconoceríamos (si entrecerráramos los ojos lo suficiente). Esos programas usaban bucles, tareas, subrutinas, pilas y otras estructuras familiares. El lenguaje de Turing era binario.
- Desde aquellos días, se han producido varias revoluciones en la programación.
- Una revolución con la que todos estamos muy familiarizados es la **revolución los lenguajes**.



Paradigmas de programación / Lenguajes de programación

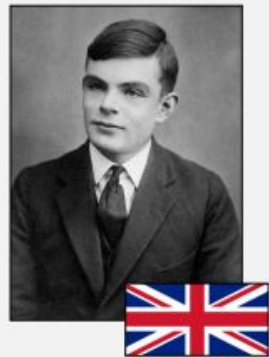


Qué restricciones tengo?

- Lenguaje de programación?
- Paradigma de programación?
- Framework?

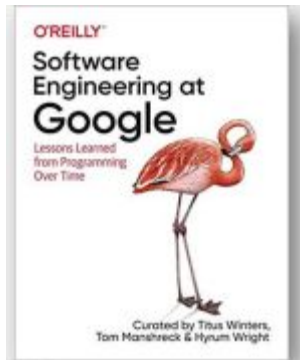
La arquitectura de software empieza con código.

- En 1938, Alan Turing sentó las bases de lo que se convertiría en programación de computadoras. Para 1945, Turing estaba escribiendo programas reales en computadoras reales en código que reconoceríamos (si entrecerráramos los ojos lo suficiente). Esos programas usaban bucles, tareas, subrutinas, pilas y otras estructuras familiares. El lenguaje de Turing era binario.
- Desde aquellos días, se han producido varias revoluciones en la programación.
- Una revolución con la que todos estamos muy familiarizados es la **revolución los lenguajes**.



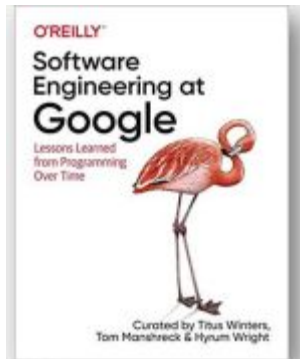
Cuál es el rol de la Ingeniería de Software en este contexto?

- *Software Engineering* is programming *Over Time*
- *Fundamental principles*
 - *Time and change: How code will need to adapt over the length of its life.*
 - *Scale and growth: How an organization will need to adapt as it evolves.*
 - *Trade-offs and Costs: How an organization makes decisions, based on the lessons of Time and Change and Scale and Growth.*



Cómo hacer “buen” software?

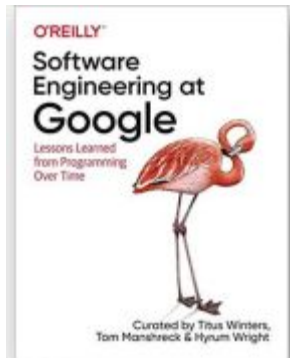
- **¿Cuánto tiempo le toma a una persona construir una pieza de software que funcione?**
 - Hoy en día no toma mucho tiempo ni habilidades desarrollar una pieza de software que funcione (incluso los niños en los colegios ya lo hacen).
- **¿Qué diferencia entonces a los programadores “amateur” de los programadores que estudian por ej: ingeniería de software?**
 - La diferencia es que hacer una pieza de código que funcione “bien” una vez no es muy difícil. Pero hacer esa pieza de código de la “manera correcta” es una cuestión totalmente diferente. Hacer que el software sea correcto es difícil toma tiempo.





Cómo hacer “buen” software?

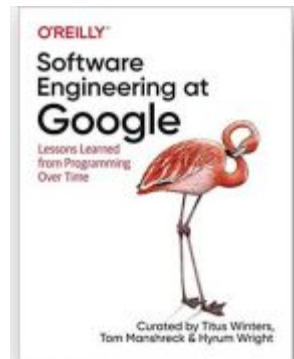
- Se requiere un alto nivel de disciplina y habilidades.
- Se requiere un nivel de compromiso y dedicación que la mayoría de los programadores nunca soñamos que necesitaríamos.
- Sobre todo, se necesita pasión por el oficio y el deseo de ser un profesional.





Y si se hace bien, qué pasa?

- Ahora solo se requiere una fracción de recursos humanos para crear y mantener el software.
- Los cambios son simples y rápidos.
- Los defectos son pocos.
- Se minimiza el esfuerzo.
- Se máxima la flexibilidad.
- Se reducen costos.
- Entre otros.





Recurso externo: [vídeo](#)

11 actividades típicas de una Arquitecta de Software

- Definir y balancear los atributos de calidad.
- Definir el problema desde el punto de vista de ingeniería.
- Establecer los principios de diseño.
- Garantizar cumplimiento de la arquitectura inicial.
- Mantenerse actualizado en tecnologías.
- Tiene que conocer de múltiples ambientes de trabajo.
- Analizar y proponer mejoras al proyecto.
- Administrar la deuda del sistema.
- Entiende y sabe moverse en el entorno político de la empresa.
- Da mentoría al equipo de trabajo.
- Desarrolla el software.



Un cuento de dos valores (*Uncle Bob*)

- Comportamiento y Estructura
 - Ambos se deben mantener a alto nivel
 - Los programadores usualmente se preocupan por el comportamiento
- Comportamiento
 - Las máquinas deberían tener un comportamiento específico → Trabajo del programador
 - Especificación funcional
 - Requisito
 - Interesados
 - Enfoque típico: se recibe el requisito -> se programa -> se arreglan bugs



Un cuento de dos valores

- **Arquitectura**
 - **Soft:** suave – **Ware:** producto
 - Está destinado a ser una manera sencilla de cambiar el comportamiento de una máquina
 - El software debe ser SOFT
 - Cuando un stakeholder cambia de opinión se debería atender fácilmente
 - Teniendo en cuenta la proporcionalidad y el alcance, no la forma del software
 - Scope != Shape



Un cuento de dos valores

- Qué es más importante, que funcione bien o que sea fácil de mantener?



Un cuento de dos valores

- Qué es más importante, que funcione bien o que sea fácil de mantener?
 - Qué respondería el gerente?



Un cuento de dos valores

- Qué es más importante, que funcione bien o que sea fácil de mantener?
 - Qué respondería el gerente?
- Es responsabilidad del equipo de desarrollo defender la importancia de la arquitectura sobre la urgencia del desarrollo
- Si la arquitectura va de última, siempre será más costoso el proceso de desarrollo



Los bloques de construcción → Paradigmas de programación

- 1945: los primeros programas formales en la perspectiva de Turing → Binario
- 1949: ensambladores → Evitan la traducción manual a binario
- 1951: A0, el primer compilador (Grace Hopper)
- 1953: Fortran → Cobol, PLII, Snobol, C, C++, Java, etc...

→ Los paradigmas de programación definen qué estructuras se usan!



Los bloques de construcción → Paradigmas de programación

- 1945: los primeros programas formales en la perspectiva de Turing → Binario
- 1949: ensambladores → Evitan la traducción manual a binario
- 1951: A0, el primer compilador (Grace Hopper)
- 1953: Fortran → Cobol, PLII, Snobol, C, C++, Java, etc...

→ Los paradigmas de programación definen qué estructuras se usan!

- Programación estructurada
 - El primero en ser adoptado más no inventado (Dijkstra, 1968)
 - Uso de saltos (Goto) afectan al sistema
 - Aparición de instrucciones IF/Then/Else y do/while/until



Los bloques de construcción → Paradigmas de programación

- Programación orientada a objetos
 - El segundo en ser adoptado (1966)
- Programación funcional
 - El primero en ser inventado y el último en adoptarse

Los paradigmas de programación, más allá de ayudar al programador, lo RESTRINGE

- Programación Estructurada → Transferencia indirecta de control
- Programación orientada a objetos → Transferencia directa de control
- Programación funcional → Asignaciones



Los bloques de construcción → Paradigmas de programación

- Qué tienen que ver con Arquitectura de Software?



Los bloques de construcción → Paradigmas de programación

- Qué tienen que ver con Arquitectura de Software?
 - Polimorfismo como mecanismo de cruce entre arquitecturas
 - Programación funcional e inmutabilidad como mecanismo de gestión de datos
 - Programación estructurada como fundación algorítmica de la solución



Los bloques de construcción → Programación estructurada

- Dijkstra concluyó que el desafío en el contexto era mayor que en el de la física
 - Para enfrentarlo, debía evolucionar el software hacia una ciencia
- “Programar es complejo y los programadores no lo hacen muy bien...”
 - Un programa contiene muchos y pequeños detalles
 - Si se ignoran, resultan en programas que parece que funcionan pero FALLAN de maneras inesperadas
- “Testing shows the presence, not the absence of bugs...”



Los bloques de construcción → Programación orientada a objetos

- La base de una buena arquitectura es el entendimiento y aplicación de principios de diseño orientados a objetos
 - Orientado a Objetos → Combinación entre datos y funciones
 - Qué es la Orientación a Objetos?



Los bloques de construcción → Programación orientada a objetos

- La base de una buena arquitectura es el entendimiento y aplicación de principios de diseño orientados a objetos
 - Orientado a Objetos → Combinación entre datos y funciones
 - Qué es la Orientación a Objetos?
 - Una manera de modelar el mundo?
 - Encapsulamiento, herencia y polimorfismo?



Los bloques de construcción → Programación orientada a objetos

- Encapsulamiento
 - Es importante? lo usan?
 - → Algunos lenguajes realmente no
 - → Esta idea sobrevive de la mano de los buenos comportamientos de los programadores



Los bloques de construcción → Programación orientada a objetos

- **Encapsulamiento**
 - Es importante? lo usan?
 - → Algunos lenguajes realmente no
 - → Esta idea sobrevive de la mano de los buenos comportamientos de los programadores
- **Herencia**
 - Es importante? la usan?
 - → Redefinición de un grupo de variables y funciones en un alcance específico
 - → Los programadores de C lo hacían mucho antes que la programación orientada a objetos existiera



Los bloques de construcción → Programación orientada a objetos

- **Encapsulamiento**
 - Es importante? lo usan?
 - → Algunos lenguajes realmente no
 - → Esta idea sobrevive de la mano de los buenos comportamientos de los programadores
- **Herencia**
 - Es importante? la usan?
 - → Redefinición de un grupo de variables y funciones en un alcance específico
 - → Los programadores de C lo hacían mucho antes que la programación orientada a objetos existiera
- **Polimorfismo**
 - Es importante? lo usan?

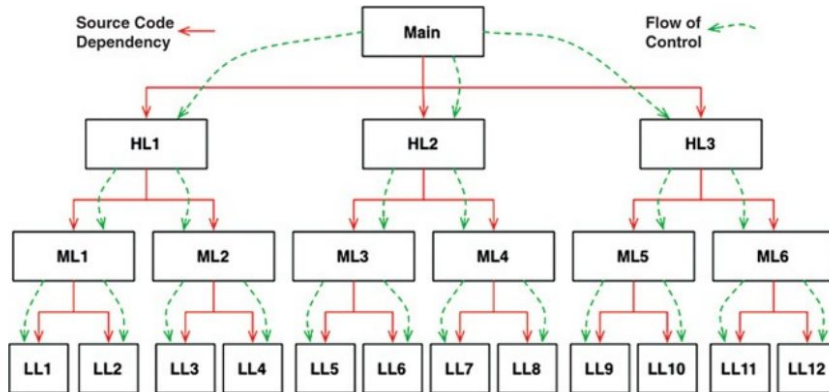


Los bloques de construcción → Programación orientada a objetos

- Polimorfismo
 - Es importante? lo usan?
 - Qué pasa cuando llega un elemento nuevo a la interfaz?
 - No depende del código externo mientras éste siga los lineamientos
 - *PLUGIN Architecture*

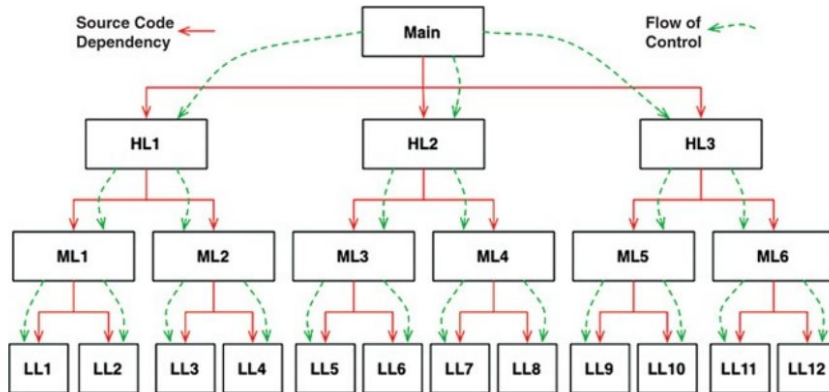
Los bloques de construcción → Programación orientada a objetos

- Polimorfismo
 - Problema típico → Las dependencias siguen el flujo

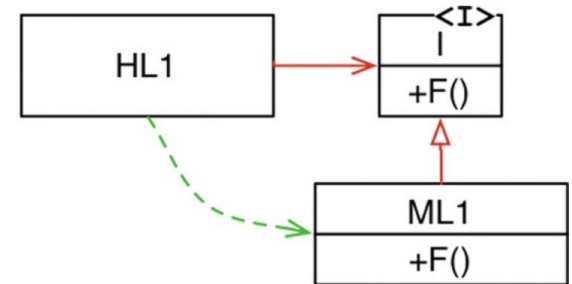


Los bloques de construcción → Programación orientada a objetos

- Polimorfismo
 - Problema típico → Las dependencias siguen el flujo

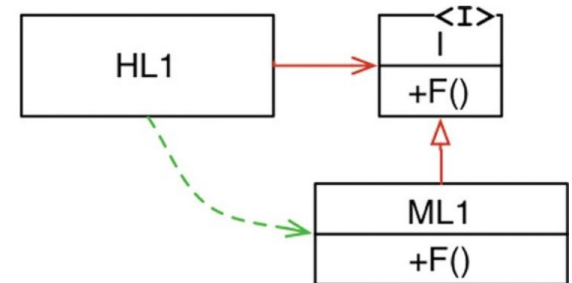


- Con polimorfismo → Inversión de dependencias



Los bloques de construcción → Programación orientada a objetos

- Polimorfismo
 - Problema típico → Las dependencias siguen el flujo
 - Con polimorfismo → Inversión de dependencias
- Cualquier dependencia del código puede ser invertida
 - Agregando interfaces entre ellas
 - Provee control sobre la dirección de las dependencias





Los bloques de construcción → Programación funcional

- Lambda Calculus - Alonzo Church, 1936
 - Variables inmutables
 - Java cambia el estado de las variables → Clojure NO
- Y por qué importa?

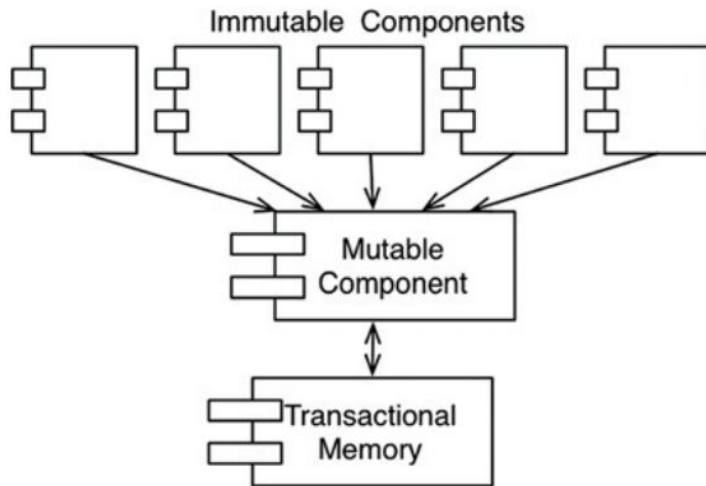


Los bloques de construcción → Programación funcional

- Lambda Calculus - Alonzo Church, 1936
 - Variables inmutables
 - Java cambia el estado de las variables → Clojure NO
- Y por qué importa?
 - TODOS los problemas de concurrencia son causadas por la MUTABILIDAD
 - → Estos problemas no puede aparecer si no se actualizan las variables

Los bloques de construcción → Programación funcional

- Un enfoque básico y práctico → Segregación de mutabilidad
 - Segregación de servicios y módulos mutables y no mutables
- *Event Sourcing*
 - Entre más memoria tenemos, más rápidos son nuestros programas y menos dependemos de estados mutables





En resumen

- Cada paradigma nos quita algo como programadores
 - Restringe algún aspecto de la manera en que escribimos código
- “Lo que hemos aprendido no es qué hacer, sino qué no hacer... El software no es una tecnología que avance rápidamente”
- Las reglas para construir software permanecen iguales
 - Pero las herramientas han cambiado mucho
 - Pero el hardware ha cambiado mucho
 - Sin embargo, la esencia se mantiene: secuencias, selecciones, iteraciones e indirecciones



Bibliografía

- Clean Architecture: A Craftsman's Guide to Software Structure and Design. Robert C. Martin.
- Software architecture, a comprehensive framework and guide for practitioners. Oliver Vogel, Ingo Arnold, Arif Chughtai, Timo Kehrer.